CE902 Machine Learning Code Review: Solved Questions and Extended Practice

Includes the 2 given questions plus 5 new longer code-snippet questions with answers, implications, and recommended fixes.

How to use this booklet

- Each question asks you to identify the main issue affecting the reliability of the model-training or evaluation process.
- The answers focus on three parts: the issue, why it matters, and how to fix it.
- The recurring themes are data leakage, optimistic bias, misuse of validation/test data, and incorrect cross-validation design.

Question 1 (given): feature selection before CV

Code snippet

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.feature_selection import SelectKBest
from sklearn.model_selection import cross_val_score

# Load dataset and create X and y
df = pd.read_csv('data.csv')
X = df.drop('target', axis=1)  # all columns except target
y = df['target']

# Feature selection
selector = SelectKBest(k=10)
X_selected = selector.fit_transform(X, y)

# Fit in cross-validation using previously selected features
model = DecisionTreeClassifier(criterion='entropy', max_depth=9)
scores = cross_val_score(model, X_selected, y, cv=5, scoring='accuracy')

print('Cross-validation_Accuracy:', scores.mean())
```

Model answer

Main issue: Feature selection is fitted on the full dataset before cross-validation, so the validation folds influence which features are retained.

Implications: This is data leakage. The reported accuracy is optimistically biased because information from each held-out fold leaks into training through SelectKBest. The selected feature set may not generalise.

Recommended strategy: Place SelectKBest inside a Pipeline and run cross-validation on the pipeline so selection is re-fitted separately inside each training fold. If k is tuned, use nested CV or a separate validation set.

Question 2 (given): tuning and evaluation on the same data

Code snippet

```
from sklearn.model_selection import cross_val_score, GridSearchCV
from sklearn.svm import SVC
import pandas as pd

# Load dataset
df = pd.read_csv('data.csv')
X = df.drop('target', axis=1)
y = df['target']

# Perform grid search within cross-validation for parameter tuning
model = SVC()
param_grid = {
    'C': [0.1, 1, 10, 100],
    'gamma': [0.001, 0.1, 1],
```

```
}
grid_search = GridSearchCV(model, param_grid, cv=5, scoring='accuracy')
grid_search.fit(X, y)

# Select best model
best_model = grid_search.best_estimator_

# Fit classifier using the best parameters
# and assess its accuracy in cross-validation
scores = cross_val_score(best_model, X, y, cv=5, scoring='accuracy')

print('Cross-Validation_Accuracy:', scores.mean())
```

Model answer

Main issue: The same data are used first to choose hyperparameters and then again to estimate performance with cross_val_score.

Implications: This causes selection bias (double dipping). The second CV score is no longer an unbiased estimate of unseen performance because model choice was already influenced by all samples.

Recommended strategy: Use nested cross-validation: outer folds for performance estimation, inner folds for GridSearchCV. Alternatively, tune on training/validation data and report once on a completely untouched test set.

Question 3 (new): preprocessing before the split

Code snippet

```
import pandas as pd
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.feature_selection import SelectKBest, f_classif
from sklearn.linear_model import LogisticRegression

df = pd.read_csv("customer_churn.csv")
X = df.drop(columns=["churn"])
y = df["churn"]

# Preprocess using the full dataset
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

selector = SelectKBest(score_func=f_classif, k=25)
X_selected = selector.fit_transform(X_scaled, y)

# Split only after preprocessing
X_train, X_test, y_train, y_test = train_test_split(
    X_selected, y, test_size=0.2, stratify=y, random_state=42
)

model = LogisticRegression(max_iter=500)
cv_scores = cross_val_score(model, X_train, y_train, cv=5, scoring="f1")
model.fit(X_train, y_train)
test_score = model.score(X_test, y_test)

print("CV F1:", cv_scores.mean())
print("Test accuracy:", test_score)
```

Model answer

Main issue: Both scaling and univariate feature selection are fitted before the train/test split.

Implications: The test set affects the mean/variance used by StandardScaler and the selected features. This leaks information and can inflate both CV and test performance, especially when feature selection is aggressive.

Recommended strategy: Split first, then use a Pipeline such as StandardScaler -> SelectKBest -> LogisticRegression. Fit the pipeline only on training data; use cross-validation on the training split and evaluate once on the untouched test split.

Question 4 (new): SMOTE before cross-validation

Code snippet

```
import pandas as pd
from imblearn.over_sampling import SMOTE
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_val_score

df = pd.read_csv("fraud.csv")
X = df.drop(columns=["fraud"])
y = df["fraud"]

# Balance the whole dataset first
smote = SMOTE(random_state=7)
X_bal, y_bal = smote.fit_resample(X, y)

model = RandomForestClassifier(
    n_estimators=300,
    max_depth=10,
    random_state=7
)

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=7)
scores = cross_val_score(model, X_bal, y_bal, cv=cv, scoring="roc_auc")

print("Mean ROC-AUC:", scores.mean())
```

Model answer

Main issue: SMOTE is applied to the entire dataset before cross-validation.

Implications: Synthetic minority samples are created using neighbours from future validation folds. This leaks fold-specific structure into training and typically produces overly optimistic ROC-AUC and recall.

Recommended strategy: Perform resampling inside each training fold only, e.g. with an imblearn Pipeline: SMOTE -> RandomForestClassifier. Then cross-validate the pipeline. Keep any final test set completely untouched and imbalanced.

Question 5 (new): shuffled CV on time-series data

Code snippet

```
import pandas as pd
from sklearn.model_selection import KFold, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.linear_model import Ridge

df = pd.read_csv("house_prices_time_ordered.csv")
df = df.sort_values("sale_date")

X = df[["trains", "crime_rate", "school_score", "interest_rate"]]
y = df["price"]

pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("model", Ridge(alpha=1.0))
])

# Standard CV with shuffling on time-ordered data
cv = KFold(n_splits=5, shuffle=True, random_state=42)
scores = cross_val_score(pipe, X, y, cv=cv, scoring="neg_mean_squared_error")

print("CV RMSE:", (-scores.mean()) ** 0.5)
```

Model answer

Main issue: A shuffled KFold is used for time-ordered data where future observations should not be available when predicting the past.

Implications: This introduces temporal leakage. Folds may train on later dates and validate on earlier dates, so the score can look much better than real deployment performance.

Recommended strategy: Use a time-aware split such as TimeSeriesSplit or a rolling-origin evaluation. Preserve chronology in feature engineering too, and fit all preprocessing only within each training window.

Question 6 (new): target encoding leakage

Code snippet

```
import pandas as pd
from category_encoders import TargetEncoder
from sklearn.model_selection import cross_val_score, StratifiedKFold
from sklearn.ensemble import GradientBoostingClassifier

df = pd.read_csv("loans.csv")
X = df.drop(columns=["default"])
y = df["default"]

cat_cols = ["postcode", "job_title", "broker_id", "channel"]

# Encode categories using the full target vector
encoder = TargetEncoder(cols=cat_cols, smoothing=5.0)
X_enc = encoder.fit_transform(X, y)

model = GradientBoostingClassifier(random_state=0)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
scores = cross_val_score(model, X_enc, y, cv=cv, scoring="roc_auc")

print("Mean ROC-AUC:", scores.mean())
```

Model answer

Main issue: Target encoding is fitted on the full dataset using all labels before cross-validation.

Implications: This is severe target leakage because every validation row contributes directly to the encoded value of its own category. The model can appear much stronger than it really is, especially for rare categories.

Recommended strategy: Fit the target encoder only inside training folds. Use a Pipeline or custom CV loop, and consider out-of-fold encoding or nested CV for tuning encoder hyperparameters such as smoothing.

Question 7 (new): using the test set for threshold tuning

Code snippet

```
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.metrics import f1_score, classification_report
from sklearn.neural_network import MLPClassifier

df = pd.read_csv("medical.csv")
X = df.drop(columns=["disease"])
y = df["disease"]

X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.3, stratify=y, random_state=1
)
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.5, stratify=y_temp, random_state=1
)

best_model = None
best_test_f1 = -1
best_threshold = 0.5

for seed in [1, 2, 3, 4, 5, 6, 7, 8]:
    model = MLPClassifier(hidden_layer_sizes=(128, 64), max_iter=500, random_state=seed)
    model.fit(X_train, y_train)

    # choose threshold using the test set
    probs_test = model.predict_proba(X_test)[:, 1]
    for threshold in np.linspace(0.2, 0.8, 13):
        preds_test = (probs_test >= threshold).astype(int)
        score = f1_score(y_test, preds_test)
        if score > best_test_f1:
            best_test_f1 = score
            best_model = model
            best_threshold = threshold

final_preds = (best_model.predict_proba(X_test)[:, 1] >= best_threshold).astype(int)
print("Final test F1:", f1_score(y_test, final_preds))
print(classification_report(y_test, final_preds))
```

Model answer

Main issue: The test set is used repeatedly for model selection and threshold tuning.

Implications: The reported test F1 is biased upward because the test set is no longer a final independent assessment; it has become part of the tuning loop. This can badly overstate real-world performance.

Recommended strategy: Use the validation set to choose random seed, architecture and decision threshold. After all choices are frozen, evaluate once on the untouched test set. If many settings are explored, nested CV is safer.